



CENTRO UNIVERSITARIO DE
CIENCIAS EXACTAS E INGENIERÍAS



Productor-Consumidor

Sistemas Operativos

Actividad 12 – Programa 6

Alumno: Quijas Contreras Victor Manuel

Código: 220429771

Carrera: ICOM **Sección:** D05

Profesor: Becerra Velázquez Violeta Del Rocío

Departamento: Ciencias Computacionales

Fecha: 26/04/2026

Índice

Índice	1
Objetivo	1
Desarrollo	1
Conclusión	2

Objetivo

Recrear la dinámica de productor-consumidor de manera simple y gráfica, con restricciones para llegar a ingresar al contenedor además de mostrar el estado actual de cada entidad en todo momento.

Desarrollo

Para el desarrollo de este programa se utilizó una lista simplemente ligada circular la cual nos permite recorrerla múltiples veces siguiendo una línea recta. Para la parte del entorno gráfico se basó utilizando la biblioteca Swing de Java que nos permite ingresar imágenes como referencia del productor, el consumidor y los consumibles.

Se utilizó como referencia a Pac-Man, que representa al consumidor, los "fantasmas" para representar al productor y las esferas para representar a los consumibles.

La lógica se basa en intentar ingresar a una de las dos entidades a la caja y realice su tarea la cantidad de veces que le corresponda. Las condiciones para poder ingresar a las cajas son las siguientes:

- Que no existe ninguna entidad adentro de la lista.}
- Si es el productor:
 - Que la lista NO este llena.
- Si es el consumidor:
 - Que la lista tenga elementos para consumir.

Para crear a ambas entidades (productor y consumidor) se generalizó los métodos que estos tienen en común y se agruparon en una superclase llamada Entidad, la cual posee todas las características que ambos tienen.

Productor-Consumidor

La ejecución del programa es sencilla. Primero se determina quien va a entrar por medio de un número aleatorio (aunque realmente no funciona) y se le prepara llamando a una función especial de la entidad que determina si puede ingresar a la lista o no. Después se registra de quien es el turno.

Cada entidad sabe lo que tiene que hacer cada ejecución por medio de la bandera de estado.

Si su estado es DESPIERTO. La entidad debe esperar hasta que se llame a la función ingresar de manera externa para que su estado cambie a INGRESANDO.

Si su estado es INGRESANDO, entonces la entidad esperará un tick de reloj para poder comenzar a trabajar.

Una vez que esperó ese tick, su estado cambia a TRABAJANDO y comenzará a ejecutar su acción. Después se verifica si en ese mismo tick terminó de realizar su trabajo, si es así, se le llama a una función externa llamada Dormitar que se encarga de asignarle un valor de sueño y cambiar su estado a DORMIDO.

Cada tick si su estado es DORMIDO, va a decrecer la cantidad de sueño que tiene e intentar despertar. Si su cantidad de sueño es cero, entonces cambiará su estado a DESPIERTO.

La función encarga de ejecutar la acción de la entidad devuelve valor. Si la entidad terminó en ese tick. Entonces la función devolverá el valor de verdadero. Sin embargo, si la entidad está dormida o aún se encuentra trabajando y su estado no es DORMIDO entonces devolverá falso.

Con esta bandera el programa es capaz de determinar si una entidad terminó su ejecución de manera exitosa y pueda actualizar las variables gráficas que muestran el progreso de ambas entidades.

Conclusión

Esta dinámica de juego se puede llevar a la práctica de manera más profesional para poder administrar memoria importante en programas más avanzados, crear la sección crítica del código de un proceso o de controlar el flujo de la memoria con el procesador. Creo que manipular este tipo de entornos genera práctica para entender de manera más lógica como resolver este tipo de problemas.